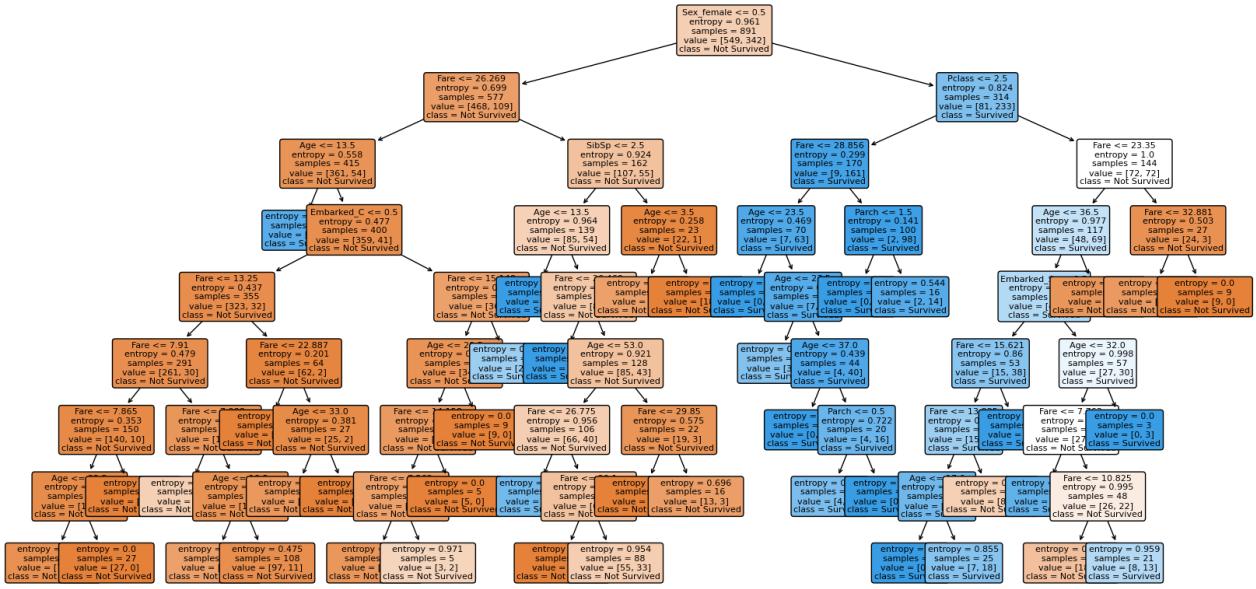
In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier


df = pd.read_csv("/Users/varunreddy/Downloads/Titanic-Dataset.csv")


X = df.drop(columns=["Survived"])
y = df["Survived"]


X = X.drop(columns=["PassengerId", "Name", "Ticket", "Cabin"], errors="ig


numeric_features = ["Pclass", "Age", "SibSp", "Parch", "Fare"]
categorical_features = ["Sex", "Embarked"]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median"))
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)


dt_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", DecisionTreeClassifier(random_state=42))
])

dt_param_grid = {
    "model__criterion": ["gini", "entropy"],
    "model__max_depth": [3, 4, 5, 6, 8, 10, None],
    "model__min_samples_split": [2, 5, 10, 20],
```

```
    "model__min_samples_leaf": [1, 2, 4, 8]
}

dt_grid = GridSearchCV(
    dt_pipeline,
    dt_param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

dt_grid.fit(X, y)

best_dt = dt_grid.best_estimator_
print("Best Decision Tree params:", dt_grid.best_params_)
print("Best Decision Tree CV accuracy:", dt_grid.best_score_)


X_processed = best_dt.named_steps["preprocessor"].fit_transform(X)
feature_names_num = numeric_features
feature_names_cat = list(
    best_dt.named_steps["preprocessor"]
    .named_transformers_["cat"]
    .named_steps["onehot"]
    .get_feature_names_out(categorical_features)
)
feature_names = feature_names_num + feature_names_cat

tree_model = best_dt.named_steps["model"]

plt.figure(figsize=(20, 10))
plot_tree(
    tree_model,
    feature_names=feature_names,
    class_names=["Not Survived", "Survived"],
    filled=True,
    rounded=True,
    fontsize=8
)
plt.title("Fine-tuned Decision Tree")
plt.show()


dt_cv_scores = cross_val_score(best_dt, X, y, cv=5, scoring="accuracy", n
print("Decision Tree fold accuracies:", dt_cv_scores)
print("Decision Tree average accuracy:", dt_cv_scores.mean())


rf_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestClassifier(random_state=42))
])
```

```
rf_param_grid = {
    "model__n_estimators": [100, 200, 300],
    "model__max_depth": [None, 4, 6, 8, 10],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 4]
}

rf_grid = GridSearchCV(
    rf_pipeline,
    rf_param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

rf_grid.fit(X, y)

best_rf = rf_grid.best_estimator_
print("Best Random Forest params:", rf_grid.best_params_)
print("Best Random Forest CV accuracy:", rf_grid.best_score_)


rf_cv_scores = cross_val_score(best_rf, X, y, cv=5, scoring="accuracy", n
print("Random Forest fold accuracies:", rf_cv_scores)
print("Random Forest average accuracy:", rf_cv_scores.mean())
```

```
Best Decision Tree params: {'model__criterion': 'entropy', 'model__max_dep
th': 8, 'model__min_samples_leaf': 2, 'model__min_samples_split': 20}
Best Decision Tree CV accuracy: 0.827185989580064
```

Fine-tuned Decision Tree



```
Decision Tree fold accuracies: [0.80446927 0.80898876 0.83146067 0.8314606
7 0.85955056]
Decision Tree average accuracy: 0.827185989580064
Best Random Forest params: {'model__max_depth': None, 'model__min_samples_
leaf': 2, 'model__min_samples_split': 2, 'model__n_estimators': 100}
Best Random Forest CV accuracy: 0.8327914129684263
Random Forest fold accuracies: [0.81564246 0.80337079 0.88764045 0.7977528
1 0.85955056]
Random Forest average accuracy: 0.8327914129684263
```

In [6]:
```
print("Random Forest average accuracy:", rf_cv_scores.mean())
```

```
Random Forest average accuracy: 0.8327914129684263
```

In [7]:
```
print("Best Random Forest params:", rf_grid.best_params_)
print("Random Forest fold accuracies:", rf_cv_scores)
print("Random Forest average accuracy:", rf_cv_scores.mean())
```

```
Best Random Forest params: {'model__max_depth': None, 'model__min_samples_
leaf': 2, 'model__min_samples_split': 2, 'model__n_estimators': 100}
Random Forest fold accuracies: [0.81564246 0.80337079 0.88764045 0.7977528
1 0.85955056]
Random Forest average accuracy: 0.8327914129684263
```

In [ ]:
```
#Random Forest performs slightly better than Decision Tree, with an avera
#This improvement is expected because Random Forest reduces overfitting b
#whereas a single Decision Tree is more prone to variance.
#Therefore, Random Forest is the better model for this task.
```